Proposed Project Structure, Coding Conventions and Open Questions

Comprehensive Management System for Radiodiagnostics and Medical Imaging (SESCAM) — Software Engineering II, Course Project 2026/2027. Team of 6. Draft v0.1 for team review.

1. Purpose and scope

This document has the three things this part of the team was asked for: a proposed Java/Maven project structure, a draft of basic coding conventions, and a list of technical questions and possible blockers.

Everything below is a proposal to discuss. Nothing is implemented here and the architecture is not decided. Whatever we agree on goes into `DECISIONS.md` through a pull request.

The workflow side (branches, pull requests, commits, CI, configuration test) is in the companion document *Development Workflow and Configuration Plan – SESCAM Radiodiagnostics*. We only point to it: §3 branches, §3.3 pull requests, §3.4 commit format, §3.7 agent rules, §3.8 definition of done, §5 configuration test.

2. Constraints and decided items

Item	What we are working with
Language and build	Java, with Maven as a multi-module project: every component is its own submodule. JDK 17 for now, to confirm in `DECISIONS.md`.
How modules talk	Only through interfaces. A module never uses another module's implementation classes. "The Engineer designs the Interfaces; the AI implements the Contracts."
Process	Unified Process (use-case driven, iterative, architecture-centric), 1 requirement = 1 use case = 1 iteration, SemVer per component, Git Flow (workflow plan §3).
Decided by the team	Similar functionalities go in the same module. Everything in English. OpenCode as the agent tool (model still open). The coordinator administers the repository.
Confirmed by the professor	Repository `2026-ISO2-A1.04` can be public. SSO, electronic health records and RIS/PACS are simulated, not connected. Fictitious data only.
Tools	Eclipse and Visual Paradigm are the course tools (licence on the Virtual Campus). Agentic environments (OpenCode, VS Code) are also allowed.
Deadlines	Theory report: **13 November 2026** (Moodle). Practical delivery: **15 December 2026** (repository evidence).

3. Proposed project structure

3.1 Repository layout

```
radio-sescam/                          repository root
├── pom.xml                            parent POM (packaging: pom)
├── mvnw, mvnw.cmd, .mvn/              Maven Wrapper: same build on every machine
├── AGENTS.md CONTEXT.md PLAN.md DECISIONS.md README.md   changed only through a PR
├── .github/workflows/ci.yml           mvn -B verify on every pull request
├── .gitattributes, .gitignore         LF endings; Java, Maven, Eclipse, VS Code
├── radio-api/                         interfaces and shared data classes
├── radio-patients/                    patients, users, roles, access
├── radio-catalogue/                   exam types, physician orders, follow-up protocols
├── radio-scheduling/                  slots, bookings, waiting lists, worklists
├── radio-reports/                     reports, incidents, surveys
├── radio-integration/                 SSO, EHR, RIS/PACS (simulated), notifications
└── radio-app/                         composition root and user interface (last iteration)

Each module:
├── pom.xml                            parent = radio-sescam, no versions here
├── CHANGELOG.md                       history of this component
└── src/
    ├── main/java/es/uclm/sescam/<module>/   api/  domain/  internal/  app/
    ├── test/java/...                  tests mirror the main packages
    └── test/resources/                fixtures with fictitious data
```

3.2 Module list (proposal, to approve in the team)

Module	What it groups	Target
`radio-api`	Interfaces and shared data classes. The only package other modules may import.	Iteration 0
`radio-patients`	Patients, users, roles (patient, physician, admin) and permissions.	Iteration 0–1
`radio-catalogue`	Exam types (modality, room, duration, preparation, priority), orders, follow-up protocols.	Iteration 1–2
`radio-scheduling`	Slots, booking, rescheduling, waiting lists, worklists, occupancy metrics.	Iteration 2–3
`radio-reports`	Reports and their review, incidents, surveys, follow-up reminders.	Iteration 3–4
`radio-integration`	Simulated adapters for SSO, EHR and RIS/PACS, plus notifications.	Iteration 4–5
`radio-app`	Wires the modules and shows the client (web and/or desktop, open question 5.1).	Transition

3.3 Packages inside a module

Package	What goes there	Rule
`es.uclm.sescam.<module>.api`	Public interfaces and immutable data classes.	The only package another module may import.
`es.uclm.sescam.<module>.domain`	Internal domain model and business rules.	Never imported from another module.
`es.uclm.sescam.<module>.internal`	Implementations, persistence, framework glue.	Never imported from another module (see 4.7).
`src/test/java` (same packages)	Unit and integration tests.	Mirror the main tree, same naming as 4.6.

Every module also has an `app` package for wiring; only `radio-app` puts modules together.

3.4 What we build in Iteration 0

- Parent POM with the module list, Java 17, UTF-8, JUnit 5 and Surefire in the parent, plus the Maven Wrapper.
- `radio-api` and one small module that delivers one use case from start to finish, with a test.
- `ci.yml`, `AGENTS.md`, `CONTEXT.md`, `PLAN.md`, `DECISIONS.md` and the wiki index.
- The rest of 3.2 is only planned: we create a module when an iteration needs it. Empty folders are not progress.

> **And the configuration test:** the generic `module-a` / `module-b` of the workflow plan §5 stay as they are until this structure is approved. Then `module-a` becomes `radio-api` and `module-b` the first slice module; the parent POM, the CI workflow and the test procedure do not change.

3.5 One open choice (not decided here)

Option	Good	Bad
A. One central `radio-api` module	Simple dependency graph; one place with every contract.	Can turn into a dumping ground; every change rebuilds all modules.
B. Each module has its own `api` package	Ownership follows the module; no artificial shared kernel.	Denser graph; the parent must pin the versions.

Both are allowed by the specification. This is an architectural decision, so it belongs to the team meeting; the choice and its reason go to `DECISIONS.md`.

4. Coding conventions (draft)

These rules are about **code**. Branches, commits, pull requests and agent rules live in the workflow plan and are only referenced here.

4.1 Naming

- Classes in `PascalCase`, methods in `camelCase`, booleans starting with `is` / `has` / `can`, constants in `UPPER_SNAKE_CASE`.
- No abbreviations (`appt` → `appointment`), no numbers in names (`Service2`), no shorthands (`mName`). Packages lowercase and singular.
- No `I` prefix on interfaces for now (Visual Paradigm uses `IFoo`) — we confirm it in the team, renaming later touches every file.
- One class, one job. No wildcard imports, no unused imports, no commented-out code.

4.2 Formatting

- 4 spaces (no tabs), max 120 columns, `K&R` braces, final newline at the end of every file.
- UTF-8 everywhere and LF endings through `.gitattributes`, so Windows, macOS and Linux produce the same diff.
- The Eclipse formatter profile lives in the repository; formatting is not a review topic.

4.3 Comments and documentation

- Every public type in an `api` package needs a Javadoc with one sentence saying what it is for (+ `@param` / `@return` when it is not obvious). Internal code needs no boilerplate.
- Comments say **why**, not what. English everywhere. A `TODO` needs an issue number: `// TODO(#42)`.

4.4 Design

- Modules talk only through `api`. Never import another module's `domain` or `internal` package.
- Shared data classes are immutable (records): no public fields, no setters.
- Do not return `null`: use `Optional<T>`. `null` only for a missing optional parameter.
- Domain exceptions are unchecked; never swallow an exception and never `printStackTrace()` — use the logger. No static mutable state, no business logic in DTOs or UI classes.

4.5 Dependencies

- Versions live only in the parent `dependencyManagement`; modules declare `groupId`, `artifactId` and scope.
- No new dependency or tool without a line in `DECISIONS.md` (workflow plan §3.7, rule 1).
- Released versions from Maven Central only. The parent pins our own modules so they cannot drift (5.3).

4.6 Tests

- JUnit 5. Unit tests are `<Class>Test`, integration tests `<Class>IT`, in the same package as the class they test.
- One behaviour per test, named after it (`rejectsBookingOutsideOpeningHours`), written Arrange–Act–Assert.
- Fictitious data only (workflow plan §3.7, rule 6). `mvn -B verify` must be green locally and in CI before review.

4.7 Rules that the build enforces (needs a team decision)

Two cheap rules are worth adding now, because they turn this document into a build failure: **maven-enforcer** in the parent (agreed Java version, no undeclared dependencies) and an **ArchUnit test** that fails if a module imports another module's `internal` package. Checkstyle and SpotBugs can wait; the specification asks us to avoid overengineering.

5. Technical questions and possible blockers

5.1 For the team

Question	Why it matters
Does everyone have JDK 17 and Maven working, or do we standardise on `./mvnw` only?	Entry requirement for the configuration test (workflow plan §5.7, step 10).
Which technology for the interface and the database?	Needed before Iteration 1, not for Iteration 0.
Which OpenCode model do we all use, and who writes the shared `AGENTS.md`?	Different rules give different code styles.
Central `radio-api` or `api` per module (option 3.5)?	It changes every POM, so decide before pushing the skeleton.
Who owns the parent POM, and who is the second repository administrator?	One administrator is a merge bottleneck and a single point of failure.
Do we add the build rules of 4.7 now, and what coverage do we require?	The Definition of Done needs a measurable number.
How do we split the theory report (13 November) among the six?	It is graded separately from the lab work.

5.2 For the professor

Question	Context
One tag per module (`radio-scheduling-1.3.0`) or a single global tag?	Already asked: `v1.3.0` alone does not say which module it is. It blocks the release branch rules.
Is 1 use case = 1 iteration strict for Block 1, or can an iteration group a few small use cases?	Taken literally, the 1:1 rule becomes a pseudo-waterfall, as the specification itself warns.
Are PlantUML diagrams imported into Visual Paradigm enough, or is the `.vpp` file required?	The specification asks for both.
Will branch protection, required checks, Actions minutes and the wiki work on the public repository?	The whole workflow depends on them.

5.3 Risks, blockers and how we avoid them

Risk / blocker	Impact	How we deal with it
The repository has 3 branches nobody recognises and no Maven project yet.	High	Coordinator audits the branches before enabling protection; keep only `main` and `develop`; push the skeleton as the one allowed first push (workflow plan §5.7.1).
Branch protection or required checks are not available on this repository.	High	Test and record it in the wiki during Iteration 0; otherwise the fallback is review policy only, which is weaker evidence for 15 December.
It builds on my machine but not in CI (6 machines, 6 setups).	High	Maven Wrapper + Temurin 17 in `ci.yml`, and "clone and run `mvn -B verify`" as the team entry checklist.
Modules with their own versions drift apart and break the final integration.	High	The parent pins every inter-module version, and every PR to `develop` builds all modules, so drift shows up in week one, not in December.
The 1 requirement : 1 use case : 1 iteration rule is read literally, and everything lands in one big integration at the end.	High	Each iteration delivers a small tested slice; nothing waits for the last iteration. This is a known lesson from previous cohorts.
Only one person can protect branches, merge and release.	Medium	Give a second member administrator rights and write the release rehearsal in the wiki.
Dates compress towards 13 November and 15 December because the domain is new to us.	Medium	Iteration 0 is only the skeleton; when a date slips we do a post-mortem and re-plan, as the specification asks.

6. What we are not deciding yet

Not decided in this document: the architecture, the final module list, the client technology, the database, the tag strategy, the interface choice of 3.5, the Java version, coverage and static analysis. All of it is open for the team meeting and `DECISIONS.md`.